

Project 2: Book Recommendations

Due Date: *Monday, November 30 (11.59PM)*

Submission: on Gradescope

Instructions

The project may be completed in groups of *up to 4 students*. If you do so, please make sure that you include everyone's full name, and that you also *select everyone's name when submitting the assignment on Gradescope*. This ensures that each group member will get a grade assigned and have access to the comments from the graders.

For this assignment, the following items need to be submitted:

1. A **single PDF** that contains your code, all results, plots, and written answers (generated from R Markdown, Jupyter, etc.).
2. The **source file** used to produce the PDF (.Rmd, ipynb, ...).

This is a **project**, *not a coding assignment*. You will not be told which popularity formula to use, which users to hold out, or how to break ties. You **will** be expected to make those choices, document them, and defend them with the ideas from class: what a baseline already explains, why neighborhood methods need a similarity and a fallback, how a held-out rating is not the same thing as a held-out user, why a similarity matrix built on all ratings cannot be reused for evaluation, and what RMSE can and cannot tell you about a top-10 list. Results in R and Python need not match; that is expected. Using a canned IBCF implementation (for example `recommenderlab`) as a stand-in for the function in part (B) is not acceptable. Sparse-matrix libraries are fine.

Project Description

The goal of this project is to make **book recommendations**. We will use the **goodbooks-10k** data set. The current release of the CSV files can be found here¹:

<https://github.com/zygmuntz/goodbooks-10k>

¹Do not use the older Kaggle copy, which has duplicate ratings.

The data contain about 6 million ratings, on a 1–5 star scale, from 53,424 users for 10,000 books, together with book metadata (`books.csv`) and a `to_read` list. Ratings in `ratings.csv` are already sorted by time, but there is *no per-user timestamp column*. The `book_id` used in `ratings.csv` and `to_read.csv` is the 1–10,000 identifier in `books.csv`, *not* the Goodreads ID.

A $10,000 \times 10,000$ item–item similarity matrix is larger than you need for this project and will be painful if computed carelessly. After loading the data, **restrict the catalog** to the N most-rated books in `ratings.csv`, with $2,500 \leq N \leq 4,000$. State N and justify it. You may also drop users with very few remaining ratings; if you do, say where you cut and why. All later parts use this [filtered rating matrix](#).

After loading, filtering, and pre-processing as needed, consider the following tasks.

- (A) Recommend the **top 10 “most popular” books**. The term “most popular” is open to interpretation. You must clearly state how you define the popularity of a book — for example a raw rating count, a mean rating with a minimum-count cutoff, a Bayesian average that shrinks toward the global mean, or something you defend. The recommendation should display the top 10 *titles* (and authors), together with the numbers behind the ranking. A ranking that is just “the 10 books with the most 5-star ratings” is acceptable only if you say so and discuss what it misses.
- (B) Make recommendations with **item-based collaborative filtering** (IBCF) by writing [your own IBCF function](#), following the steps below. Use the *full* filtered rating matrix here. Part (B) is how you inspect the method. It is *not* the matrix you will evaluate in (C).
1. Create the rating matrix R , with rows = users and columns = books in your filtered catalog. Normalize R by centering each row: subtract the row mean from each row. The matrix is *sparse*, so row means must be computed from the non-NA entries only.
 2. Compute the **cosine similarity** for all pairs of books, using the centered matrix from (1). Ignore similarities computed from fewer than 3 co-raters. You may use the $(1 + \cos)/2$ transformation so that similarities lie in $[0, 1]$. You will have NAs when a pair of books has been co-rated by 0, 1, or 2 users, or when the denominator is zero (constant centered vectors).
 3. For the similarity matrix in (2), keep only the *top 30* non-NA similarities in each row and set the rest to NA. Display the pairwise similarity values from this truncated matrix for the following books (match titles to `books.csv` carefully; several Harry Potter and Hunger Games volumes exist):
 - “The Hunger Games (The Hunger Games, #1)”

- “Harry Potter and the Sorcerer’s Stone (Harry Potter, #1)”
- “Pride and Prejudice”
- “The Iliad”
- “The Odyssey”

Write two or three sentences on whether the neighbors look like a content-based genre match, a popularity match, or something less obvious.

4. Write your own *IBCF* function.

- *Input*: `newuser`, a vector of ratings for every book in your catalog. Many entries will be NA; non-NA values are stars in $\{1, 2, 3, 4, 5\}$.
- *Output*: the top 10 book recommendations for this user, as titles. If fewer than 10 predictions are non-NA, fill the remaining slots from the popularity ranking in (A), skipping books the user has already rated. You may want to store that ranking once so you do not recompute it.
- The function should load (or receive) the truncated similarity matrix and predict ratings for unrated books with the weighted-average formula from the notes. State how you handle a book whose neighbors were also unrated by this user.

5. Test the function by displaying the top 10 recommendations, with predicted scores when you have them, for:

- One *real* user from your filtered matrix who has between 30 and 80 ratings. Report the `user_id` and a short sketch of what they have already rated (a handful of titles is enough).
- A hypothetical user who rates “Harry Potter and the Sorcerer’s Stone (Harry Potter, #1)” with 5 *stars* and “Pride and Prejudice” with 5 *stars*, and nothing else.
- One hypothetical user *per group member*. Each member picks their favorite book, rates it with 5 *stars*, and rates nothing else. The title must appear in your filtered catalog (one of the N most-rated books). If someone’s actual favorite is not on that list, they pick the closest book that is, and say so. Report each member’s name, the exact title as it appears in `books.csv`, and that member’s top 10 list.

Compare each list, briefly, to the top 10 from (A). How much overlap is there, and what does that say about popularity bias in IBCF?

- (C) **Evaluation.** The lists in (B) show what IBCF recommends. They do not show whether it is more accurate than a simple baseline. Parts (C1)–(C4) use the *same filtered catalog* as (A)–(B), but a *different* rating matrix.

1. *Split ratings, not users.* Start from the filtered matrix R of part (B). Among users who have at least 10 observed ratings, randomly select 20% of *that user's* observed entries (at least one) and move them to a test set. Replace those test entries in R by **NA**. Call the result R_{train} . The test set is a list of triples (u, j, r_{uj}) that you no longer treat as observed.

Because `ratings.csv` has no per-user timestamp, drawing the test ratings uniformly at random within each user is the default. If you instead cut the file using its global time order, say so and defend the choice.

2. *Refit IBCF on R_{train} only.* The similarity matrix S from part (B) was built using every observed rating, including the ones that are now test data. **Do not use that S here.** On R_{train} — that is, after the test entries have been set to **NA** — recompute, in order:
 - the row means (from the remaining non-**NA** entries only),
 - the centered cosine similarities,
 - the rule that drops pairs with fewer than 3 co-raters,
 - the top-30 truncation.

Then pass R_{train} and this new S into the IBCF function from (B)4. If one object S in your code is used both for the lists in (B) and for the numbers in (C), the test ratings have leaked into the neighbor graph. Write one sentence in the report confirming that S was recomputed from R_{train} .

3. *Rating error.* Using only the held-out triples from (C)1, report **RMSE** (MAE optional) for at least three predictors:
 - a popularity or global-mean baseline consistent with part (A),
 - a user-mean baseline (or $\mu + a_i + b_j$, if you implement it),
 - IBCF, with the S fit on R_{train} .

If IBCF cannot predict a test pair (u, j) because u has no rated neighbor of j in R_{train} , fill that prediction with the popularity fallback from (B)4. Do not drop the pair. Report how many test pairs used the fallback.

4. *Ranking error.* RMSE scores a predicted number of stars. A recommender returns a list. Also report **Precision@10** (Recall@10 or NDCG@10 optional). For each test user, take the 10 books IBCF ranks highest among books that user did *not* rate in R_{train} . A book is relevant if that user's held-out rating for it is 4 or 5. Optional: treat books on that user's `to_read` list as additional relevant items and say whether Precision@10 changes.

5. In a short paragraph: did IBCF beat the baselines on RMSE? On Precision@10? If the two metrics disagree, which one would you trust for a “top 10 books for you” product, and why?